React Assignment Sheet 3-Day Incremental Case Study: Smart Hostel Maintenance Portal

Student-ready assignment document with visual UI reference screens

Scenario
A hostel wants a small web portal where students can raise maintenance complaints and the hostel admin/warden can track, manage and update those requests.

Project Continuity Rule
This is **one single project** to be developed in three incremental phases. Day 2 must continue from the Day 1 implementation, and Day 3 must continue from the Day 2 implementation.

Mandatory Technologies
React Functional Components, React Router DOM, Axios, Bootstrap, Context API, useEffect, useMemo, Custom Hooks, json-server, Formik, Yup

Business Need: Hostel students face issues such as electrical faults, plumbing leaks, internet problems and room maintenance requests. Currently these complaints are shared informally and are difficult to track. The hostel wants a dashboard-based application to manage complaints properly.

General Instructions

- Build **one common React application** and enhance it day by day instead of creating separate mini-projects.
- Use **json-server** as the backend and consume all data through **Axios**. Do not hardcode complaint data in components.
- Use **Bootstrap** for layout, forms, buttons, cards, tables and responsive design.
- Use **Formik + Yup** for form handling and validation in all user input forms.
- Use proper folder structure for **components**, **pages**, **context**, **hooks** and **services**.
- Add meaningful validations, loading states, empty states and error handling wherever applicable.

API Setup Requirement (json-server)

Create a **db.json** file with at least the following collections:

- **users** - student and admin records with id, name, email, password, role and room number
- **requests** - complaint records with id, title, description, category, room number, studentId, studentName, priority, status and createdAt
- **categories** - complaint category master data such as Electrical, Plumbing, Cleaning, Furniture and Internet

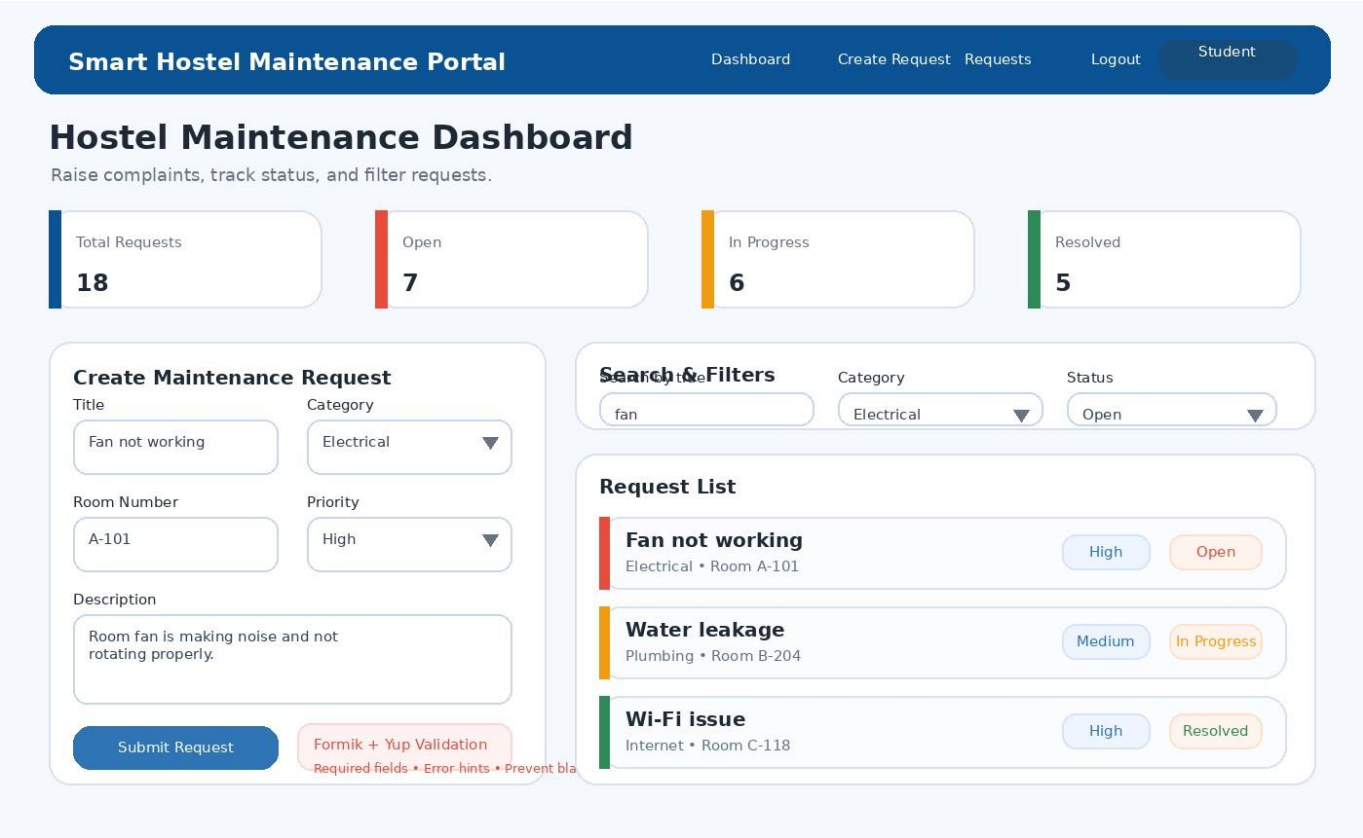
Expected endpoints: GET /users, GET /users?email=...&password;=..., GET /requests, GET /requests/:id, GET /requests?studentId=..., POST /requests, PATCH /requests/:id, GET /categories.

Suggested commands: npm install, npm install axios react-router-dom bootstrap formik yup, npx json-server --watch db.json --port 5000, npm start

DAY 1 - Build the Basic Complaint Management Dashboard

Goal: Create the first working version of the application where hostel students can raise maintenance requests, view all requests, search/filter them and view basic complaint statistics.

Reference UI Screenshot



Functional Requirements

- **Dashboard Page**: Create a page titled **Hostel Maintenance Dashboard**.
- **Summary Cards**: Display cards for **Total Requests**, **Open**, **In Progress** and **Resolved**. These counts must update dynamically whenever data changes.
- **Create Request Form**: Build a complaint form with fields for **Title**, **Description**, **Category**, **Room Number** and **Priority**. On successful submission, the request must be stored in json-server with default status as **Open**.
- **Request Listing**: Display all complaints in a card or list view showing title, category, room number, priority, status and created date.
- **Search and Filter**: Provide search by title, filter by category and filter by status.
- **React Communication**: Clearly demonstrate parent-to-child and child-to-parent communication.

Mandatory Validation Requirement (Formik + Yup)

- **Complaint Form must be created using Formik**. Use controlled form submission through Formik handlers.
- Use **Yup** schema validation for all fields.
- Validation rules should include: required title, required description, required category, required room number, required priority.

- Add user-friendly validation messages below the relevant input elements.
- Disable blank submissions and display a success confirmation after a valid complaint is created.

Technical Expectations

- Use **useEffect** to fetch categories and requests when the page loads.
- Use **Axios** for GET and POST API calls.
- Use **Bootstrap** cards, grid, buttons, form controls and spacing classes for a clean layout.

Deliverable

A working single-page complaint dashboard with a validated request form, complaint list, search/filter functionality and summary cards — all connected to json-server.

Suggested Collections

users


```
[
  {
    "id": 1,
    "name": "Aarav",
    "email": "aarav@student.com",
    "password": "1234",
    "role": "student",
    "roomNo": "A-101"
  },
  {
    "id": 2,
    "name": "Diya",
    "email": "diya@student.com",
    "password": "1234",
    "role": "student",
    "roomNo": "B-204"
  },
  {
    "id": 3,
    "name": "Warden Meera",
    "email": "warden@hostel.com",
    "password": "admin123",
    "role": "admin",
    "roomNo": "-"
  }
]
```

Requests


```
[
  {
    "id": 1,
    "title": "Fan not working",
    "description": "Room fan is not rotating properly",
    "category": "Electrical",
    "roomNo": "A-101",
    "studentId": 1,
    "studentName": "Aarav",
    "priority": "High",
    "status": "Open",
    "createdAt": "2026-06-11"
  },
  {
    "id": 2,
    "title": "Water leakage",
    "description": "Leakage in bathroom tap",
    "category": "Plumbing",
    "roomNo": "B-204",
    "studentId": 2,
    "studentName": "Diya",
    "priority": "Medium",
    "status": "In Progress",
    "createdAt": "2026-06-11"
  }
]
```


Categories


```
[
  { "id": 1, "name": "Electrical" },
  { "id": 2, "name": "Plumbing" },
  { "id": 3, "name": "Cleaning" },
  { "id": 4, "name": "Furniture" },
  { "id": 5, "name": "Internet" }
]
```


Required Endpoints

With json-server

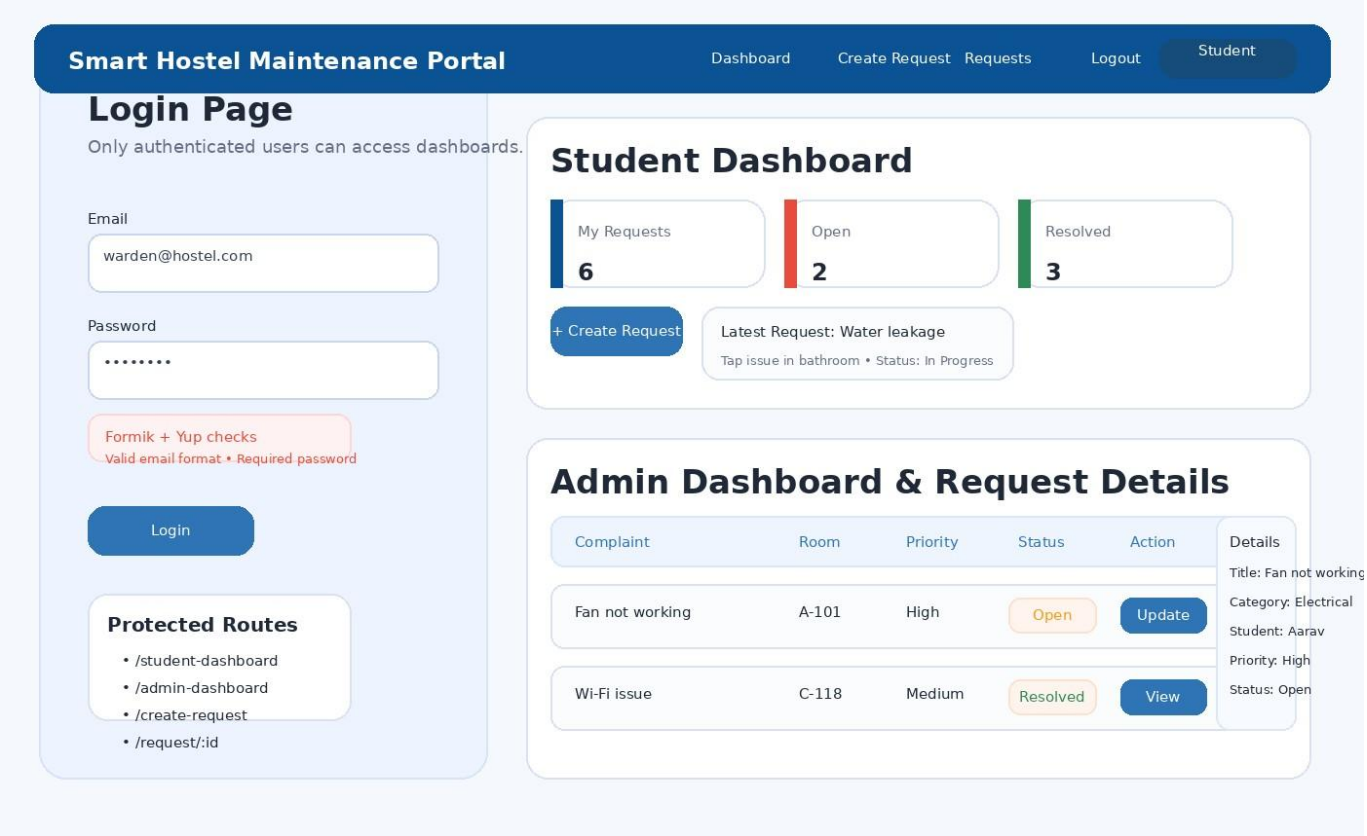
- `GET /users`
- `GET /users?email=value&password=value`
- `GET /requests`
- `GET /requests/:id`
- `GET /requests?studentId=1`
- `POST /requests`

- `PATCH /requests/:id`
- `DELETE /requests/:id` *(optional for admin)*
- `GET /categories`

DAY 2 - Add Login, Routing, Protected Pages and Role-based Flow

Goal: Continue the same Day 1 project and convert it into a multi-page application with login, role-based dashboards, protected routes and detailed complaint management.

Reference UI Screenshot



Functional Requirements

- **Login Page**: Create a login page where users enter email and password.
- **Auth Context**: Use **Context API** to store the logged-in user object, authentication status, login function and logout function.
- **Routing**: Create routes such as **/login**, **/student-dashboard**, **/admin-dashboard**, **/create-request** and **/request/:id**.
- **Protected Route**: Restrict all dashboard and request-related pages to authenticated users only. If a user is not logged in, redirect to the login page.
- **Student Dashboard**: After student login, show only the complaints raised by that student.
- **Admin Dashboard**: After admin login, show all complaints and allow the admin/warden to update complaint status.
- **Request Details Page**: Create a route using **/request/:id** and display full complaint details.

Mandatory Validation Requirement (Formik + Yup)

- **Login Form must use Formik + Yup**. Validate required email and password fields.
- Email must be validated using proper email format.
- Password must be validated as mandatory.
- **Create Request Form from Day 1 must continue to use Formik + Yup** in this multi-page version also.

Technical Expectations

- Use **Axios** to validate login using the users endpoint.
- Store logged-in user data in Context API and use it throughout the app.
- Use route parameters to fetch individual complaint details.
- Use **PATCH** request for admin status update.

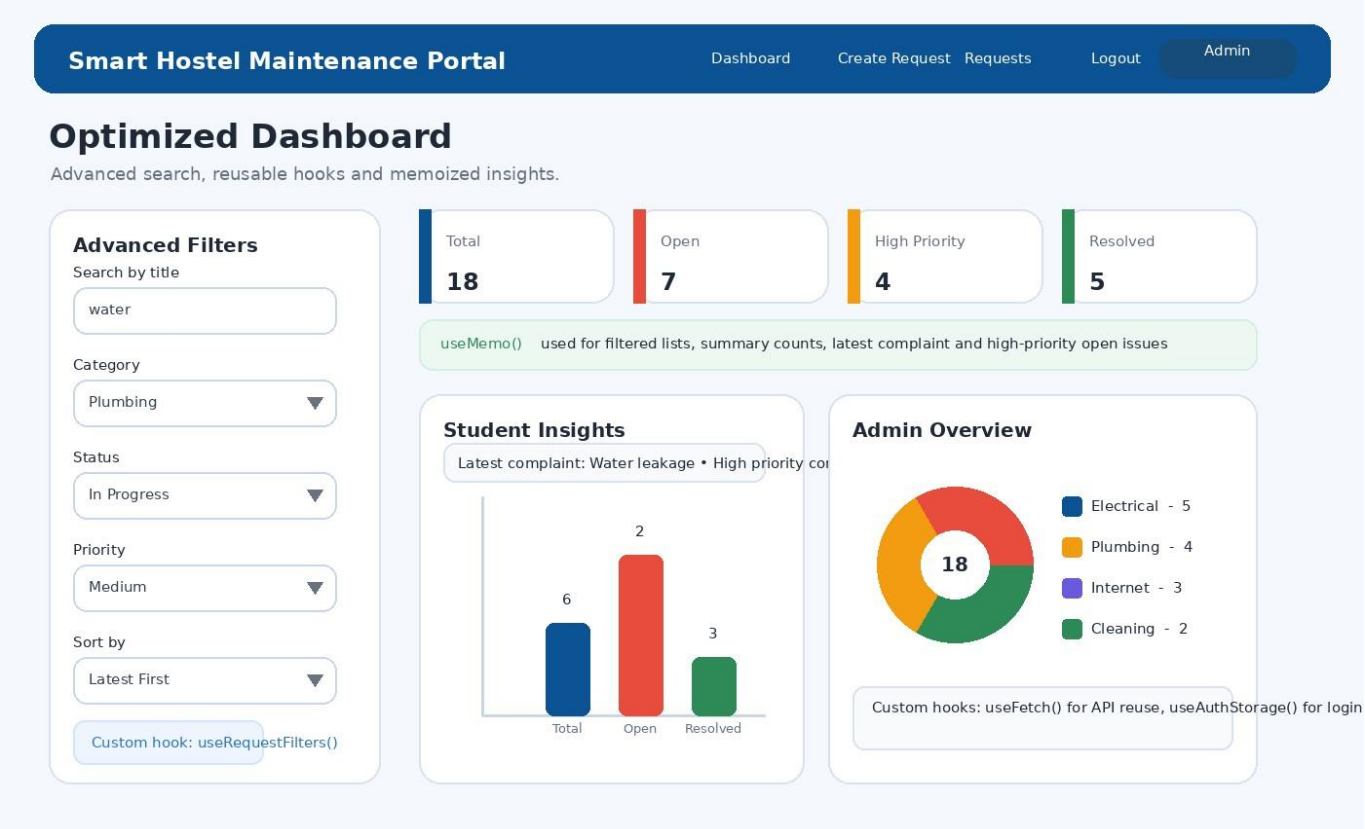
Deliverable

A working multi-page complaint portal with login, Context API, protected routing, student and admin dashboards, complaint detail page and status update feature.

DAY 3 - Add Custom Hooks, useMemo Optimization and Advanced Dashboard Features

Goal: Continue the same Day 2 project and improve reusability, optimization and user experience by introducing custom hooks, memoized calculations and enhanced filtering/sorting.

Reference UI Screenshot



Functional Requirements

- **Custom Hooks**: Create at least two custom hooks such as **useFetch**, **useRequestFilters** or **useAuthStorage**.
- **Advanced Search and Sort**: Support search by title, filter by category, filter by status, filter by priority, sort by latest, sort by oldest and sort alphabetically by title.
- **useMemo Optimization**: Use **useMemo** to compute filtered request lists, dashboard summary cards, high-priority open requests count and student-specific complaint statistics.
- **Student Insights**: Show total complaints raised by the logged-in student, open complaints, resolved complaints, latest complaint and high-priority complaints.
- **Admin Overview**: Show total complaints, open/in-progress/resolved counts, category-wise counts and number of high-priority open issues.
- **Refactoring**: Reuse common components such as RequestCard, FilterBar and SummaryCards across dashboards.

Mandatory Validation Requirement (Formik + Yup)

- All forms introduced in the project must continue to use **Formik + Yup**. Existing validations must remain intact after refactoring.
- Provide loading indicators, no-data messages and graceful handling of API errors.
- Persist login state if using localStorage so the user remains authenticated after refresh (recommended).

Technical Expectations

- Use custom hooks for reusable API/data logic.
- Use **useMemo** for optimized calculations.
- Keep the Bootstrap UI consistent while improving dashboard clarity and usability.

Deliverable

A cleaner and more optimized portal with reusable hooks, advanced filtering/sorting, memoized dashboard calculations and improved code organization.

Evaluation Checklist

- **Functionality**: Required Day 1, Day 2 and Day 3 features work correctly without breaking earlier functionality.
- **Formik + Yup Usage**: All major forms use Formik for form state and Yup for validation.
- **API Integration**: Proper use of json-server endpoints through Axios.
- **Routing and Security**: Protected routes and role-based page access work correctly.
- **React Concepts**: Appropriate use of state, props, Context API, parent-child communication, useEffect, useMemo and custom hooks.
- **UI Quality**: Bootstrap-based layout is clean, responsive and easy to use.
- **Code Quality**: Components are reusable, folder structure is neat and naming is meaningful.

Submission Requirements

- Submit the complete React project folder.
- Include **db.json** used for json-server.
- Include a short README with project setup steps and login credentials for student and admin users.
- Ensure the application runs without manual code changes after installation.

Important Note for Students

This assignment is expected to be completed as a single evolving project. Do not start a new project each day. Your Day 2 implementation must continue the Day 1 codebase, and your Day 3 implementation must continue the Day 2 codebase.

The UI screenshots in this document are reference visuals to help you understand the expected layout and user flow. You may improve the design, but the required features and validations must remain intact.